WIRELESS MIDDLEWARE

Wireless middleware is an intermediate software component that is generally located on a wired network between the wireless device and the application or data residing on a wired network.

The purpose of the middleware is to increase performance of applications running across the wireless network by serving as a communication facilitator between components that run on wireless and wired devices (Wireless Nets).

It serves as a communication facilitator by addressing the numerous ways in which communication can fail among components in a distributed application. Sources of communication failure, among many others, include a component going off line voluntarily but unexpectedly, a component unexpectedly terminating, a component failing to respond to a request in a reasonable amount of time, and communication being severed in the midst of a request.

How it Works

Wireless middleware usually consists of client and server software. The client portion resides on the mobile computer and accepts messages from applications on the mobile computer. It reformats these messages and forwards them across the wireless network using application-layer protocols optimized for wireless communications. The messages reach the middleware server, which typically resides on the destination LAN. The middleware server functions as a gateway to other servers and hosts on the LAN, acting as a proxy for the mobile computer.

Middleware performs the following functions, though specific details will vary depending on the actual middleware.

- Isolates the application from connectivity issues such as intermittent connections and varying throughput.
- Minimizes the amount of data sent over the wireless connection.
- Reduces the number of back-and-forth messages required to complete a transaction.
- Queues messages when a connection is not available.

- Provides a consistent API regardless of the underlying network. (An **application programming interface** (**API**) is a computing interface that defines interactions between multiple software or mixed hardware-software intermediaries. It defines the kinds of calls or requests that can be made, how to make them, the data formats that should be used, the conventions to follow, etc.)

TYPES OF WIRELESS NETWORKS

Currently there are two distinct types of wireless networks that can be categorized by their transmission range and that differ significantly in the latency and bandwidth characteristics of the transmission.

- Fixed Wireless Networks

Fixed Wireless Networks use wireless transmitters with a relatively short transmission distance and normally correspond to one of the IEEE 802.11 standards. Networks of this type have many of the characteristics of fixed wire networks, such as low latency, high bandwidth, and relatively good reliability.

- Highly Mobile Wireless Networks

These wireless networks are used predominately to serve highly mobile clients such as mobile cellular phones or satellite communications. Networks of this type have extremely high latency, relatively low bandwidth, and greater unreliability, as any number of interruptions can occur to disrupt the communication with the client. As this type of network is increasingly becoming used for M-commerce, specialized middleware is required to cater to the additional challenges.

Assignment

Describe the standard middleware that is normally employed in these types of networks

CHALLENGES FOR WIRELESS MIDDLEWARE

Wireless middleware is presented with many challenges by the very nature of the wireless environment. These challenges are not adequately addressed by standard data communication methods that were designed for wired environments.

One of the main advantages of middleware is to allow applications to run on multiple platforms without the need to rewrite them. As such, wireless middleware should support heterogeneous devices, permit applications to be ported from one device to another, and, in addition, handle the limitations of mobile devices, among which are the small amount of memory and the lack of processing power.

Wireless middleware on mobile devices has some distinct issues not present in traditional fixed line systems.

- Disconnected Operation

Wireless communication devices tend to lose and regain network connectivity much more often than non-mobile applications. The middleware must be able to cope with intermittent communication links and implement software layers that ensure the delivery of important data between the server and the wireless device.

- Resource Constraints

Mobile applications must be optimized aggressively for small ROM and RAM footprints, as well as for low usage of CPU cycles and battery power.

- Multiple Bearers

Wireless applications are written to perform on many different networks.

A bearer in a wireless network is a transport mechanism of which wireless bearers could be SMS, GPRS, Infrared, Bluetooth, or HTTP. An application written for one bearer typically needs to undergo substantial modifications in order to run on another bearer.

Internet applications only need to support HTTP or TCP/ IP.

- Heterogeneous Software

There are a number of operating systems that are prevalent in the wireless device market. Because of the memory restrictions placed on these systems, they do not possess all the features of server operating systems.

Thus, it is often difficult to ensure that the middleware can effectively run on all the required hardware devices.

Often, features that are available on one device are not available on another device with a different operating system.

- Security

Security is a key concern in a mobile environment. Identification, authentication, and data encryption must be embedded into wireless middleware applications.

- Scalability

As a result of the proliferation of wireless devices, scalability of applications can easily grow to hundreds of thousands of client systems. Ensuring the scalability of applications to this number of clients presents a major problem.

- Deployment and Management

Deploying applications to many clients, managing them, supporting them, and ensuring their integrity and support when they are not online, present another major challenge to the middleware.

Commercial Significance of Wireless Middleware

The development of application software such as M-Commerce applications and client/server applications should not be designed so they are independent of the network over which they are deployed. This ensures that applications do not require to be rewritten for new technologies, and the applications developer and business enterprise can concentrate on the business benefits of the application and not the technology on which they reside. For example, a stock ordering application for a manufacturing concern should integrate with the manufacturing schedule in the

corporate head office. The speed of taking the orders, scheduling them, and getting them in the production schedule is what is important to the manufacturing company.

Wireless middleware provides this network transparency and enables wired applications to be ported to wireless environments with the minimum of software modification.

Wireless Network Requirements/ Common Features of Middleware Products

In order to meet the needs of the applications running on them, typical wireless middleware solutions have the following basic requirements:

• Intelligent restarts

The middleware should neither raise communication errors nor lose data when a mobile device loses network coverage. It should incorporate a recovery mechanism that detects when a transmission has been cut, and when the connection is reestablished, the middleware resumes transmission from the break point instead of at the beginning of the transmission.

• Store-and-forward messaging

Middleware often performs message queuing to ensure that users disconnected from the network will receive their messages once the station comes back online. When the station comes back online, the middleware sends the stored messages to the station.

- Small Footprint

It is particularly important that the messaging client library, stored in the wireless device should have a small memory footprint (ROM and RAM).

- Open bearer Model

The middleware should offer the same set of communication abstractions on top of various wireless bearers. Modern wireless communications use GSM, GPRS, and possibly 3G mobile communication protocols. This allows applications to be developed once and operate on top of various bearers.

- Multi-Platform Language Availability

The Application Program Interface (API) should be compatible with computer languages available on multiple platforms and operating systems. This allows applications to be adapted for various platforms more easily.

- Security

Security is essential in a wireless system, particularly for applications such as M-Commerce, which are becoming increasingly widespread.

As wireless communications cannot be physically secured, the needs of wireless access present a new series of challenges.

Wireless access to enterprise systems puts not only the client device, but also the data, well beyond the physical control of the organization.

Sniffing of data traffic can be done without any risk of detection over a much wider range of locations. Furthermore, the client device, in the case of a cell phone or PDA, is even easier to steal than a laptop computer with the additional loss of security. Compromise of the wireless client thus poses a double threat to data: the remote access to data that the device enables, and immediate access to the downloaded data which is stored within it. The wireless middleware should incorporate a security mechanism.

Thus, access control, identification, authentication, and end-to-end data encryption should be provided by the middleware. Providing these facilities in the middleware and not in the application dramatically simplifies the development of secure mobile solutions.

- Scalability

The middleware must support scalability, as many thousands of clients could be connected. It must not become overloaded when repeatedly sending messages, and must handle increasing congestion resulting from disconnect clients.

In addition, it must be able to identify and delete messages that can never be delivered perhaps due to a wireless device becoming broken or stolen.
Intelligent restarts are required in order to enhance the robustness of the distributed system that is built using the middleware. A robust distributed system must detect failures, reconfigure the system so that computations may continue, and recover when a link is repaired.

- Optimization techniques

Many middleware products include data compression at the transport layer to help minimize the number of bits sent over the wireless link. Some implementations of middleware use header compression, in which mechanisms replace traditional packet headers with a much shorter bit sequence before transmission.

- Data bundling

Some middleware is capable of combining (bundling) smaller data packets into a single large packet for transmission over the wireless network. This is especially beneficial to help lower transmission service costs of wireless WANs. Because most wireless data services charge users by the packet, data bundling will result in a lower aggregate cost.

- Operational support mechanisms

Some middleware products offer utilities and tools to monitor the performance of wireless end-user devices, enabling MIS personnel to better troubleshoot problems.

- Application-development tools

Some middleware packages also include application-development tools.

Many companies offer wireless middleware products. The selection of a preferred vendor depends on wireless network requirements, such as end-user devices and end systems.

Messaging Middleware

These basic requirements of wireless middleware can be met by messaging middleware.

Messaging middleware allows general-purpose messages to be exchanged in a client/server system using message queues.

Applications communicate over networks by putting messages in queues and getting messages from queues.

This messaging and queuing allow clients and servers to communicate across a network without being linked by a private, dedicated, logical connection. The clients and servers can run at different times.

All communication is by putting messages on queues and by taking messages from queues. It is used when a distributed application can tolerate a certain level of time-independent responses. For example, nomadic client/server systems can accumulate outgoing transactions in queues and do a bulk upload when a connection can be established with an office server.

Since most of the commercial middleware products are based on the Java Messaging System (JMS).

JMS, developed by Sun Microsystems, Inc., is part of the Java2 Enterprise Edition (J2EE) platform. JMS is an Application Programming Interface (API) for accessing Messaging applications in Java Programs. The JMS API is not specifically intended for the wireless domain. Traditional JMS was designed to allow Java applications to communicate with existing wireline messaging systems. As a result, a full JMS implementation is too "fat" for wireless devices because low power consumption and a smaller memory footprint are required. However, JMS has become the "de facto" standard as an API for wireless middleware, and most wireless middleware products are based on it. The JMS API allows applications to create, send, receive, and read messages which can arrive asynchronously, which means that the client does not have to specifically request messages in order to receive them.

Note: Read more on JMS API and JMS API Architecture

Future Trends

As wireless devices become increasingly powerful and more accepted, wireless middleware will have to play an increasing role in ensuring that applications run efficiently in this environment. Currently, the versions of wireless middleware being used are, at best, second-generation versions; however, work is being undertaken in a number of areas.

- Network Protocols

Alternative network protocols suitable for wireless communication are being analyzed. This will lead to faster communications between the wired and wireless devices.

- Application Communication Paradigms

As wireless technology becomes more available, developers of distributed applications are becoming more interested in how that technology affects the performance of their systems. Thus, the efficiency of different paradigms of communication, such as publish/subscribe, is being investigated.

- Efficiency of Wired Protocols

As most distributed applications on the Internet are using protocols such as SOAP and XML, work is being undertaken to optimize these protocols for wireless communications.

Alternative Middleware Products

Currently, Java (JMS, in particular) is effectively the only platform suitable for mobile communications. As the market for wireless applications becomes larger and more profitable, there will be more middleware products vying for a share. Thus, there will be greater competition and more choice for the applications developer. Some of these middleware products could permit applications for wireless ad hoc networks such as peer-to-peer computing, which could easily be developed.

As a result of this work, wireless applications will be developed, integrated into the enterprise, and executed with the efficiency and reliability that is expected from wired applications.